

SISTEMA PDV CASHLESS PARA BARES

NFC / Pulseira • Maquininhas de Cartão • Conforme SEFAZ

Plano técnico completo para desenvolvimento no Cursor IDE

Versão 1.0 — Julho 2026

Sumário

1. Visão Geral do Produto

Este documento define o plano de desenvolvimento completo de um sistema PDV (Ponto de Venda) Cashless para bares e eventos. O sistema funciona de forma semelhante ao Zig.io: o cliente carrega créditos em uma pulseira ou cartão NFC e consome no bar sem precisar de dinheiro ou cartão bancário a cada pedido.

1.1 Objetivos Principais

- Pulseira/Cartão NFC como mídia de pagamento pré-pago (cashless)
- Integração com maquininhas Smart POS Android (Pague Seguro, Stone, Cielo, PagSeguro, Mercado Pago, Getnet, Rede, SumUp)
- Operação offline-first: o PDV funciona mesmo sem internet
- Painel web de gestão: relatórios, estoque, recarga de saldo
- Emissão de NFC-e conforme SEFAZ

1.2 Fluxo de Negócio (Como o Zig.io Funciona)

1. Cliente chega ao evento e realiza recarga (dinheiro, cartão, PIX) numa totem ou caixa
2. Sistema vincula saldo à pulseira NFC do cliente
3. No bar, o bartender usa o Smart POS para lançar os itens do pedido
4. Cliente aproxima a pulseira no leitor NFC do Smart POS
5. Sistema verifica saldo, debita e imprime o comprovante
6. No encerramento, cliente solicita devolução do saldo remanescente

1.3 Diferença do Sistema Tradicional

Aspecto	PDV Tradicional	PDV Cashless (este sistema)
Pagamento	Cartão bancário / Dinheiro	Pulseira/Cartão NFC pré-pago
Velocidade	30-60 segundos por transação	3-5 segundos por transação
Controle de consumo	Sem controle individual	Histórico completo por pulseira
Fila	Alta (pagamento demorado)	Mínima (toque e vai)
Devolução de saldo	N/A	Automática via PIX/Dinheiro
Risco de troco errado	Alto	Zero

2. Arquitetura do Sistema

O sistema é composto por quatro camadas principais que se comunicam via API REST e WebSockets.

2.1 Visão Geral da Arquitetura

Camada	Tecnologia	Descrição
Smart POS App	Android (Kotlin/Java) ou React Native	Aplicativo rodando diretamente na maquininha
Backend API	Node.js + NestJS + PostgreSQL	Servidor central na nuvem
Painel Admin	React PWA	Gestão, relatórios, recarga de saldo
Banco local (offline)	SQLite (Room no Android)	Cache local para operação sem internet
Sincronização	Supabase Realtime / WebSocket	Sync bidirecional online/offline
Motor Fiscal	Integrador NFC-e	Emissão de notas fiscais

2.2 Stack Tecnológica Detalhada

Backend

- Runtime: Node.js 20 LTS
- Framework: NestJS (TypeScript)
- Banco de dados: PostgreSQL 16 (Supabase)
- Cache: Redis (filas de sincronização e sessões NFC)
- Auth: JWT + Supabase Auth
- Realtime: Supabase Realtime (WebSockets)
- Deploy: Railway.app ou Render.com (ou VPS)

Aplicativo Smart POS

- Linguagem: Kotlin (Android nativo) — melhor compatibilidade com os SDKs das maquininhas
- Banco local: Room (SQLite) + Outbox Pattern para sync offline
- NFC: Android NfcAdapter (leitura de UID ISO 14443-A/B)
- UI: Jetpack Compose (interface moderna e rápida)
- Rede: Retrofit2 + OkHttp3
- Build: Gradle (APK ou AAB assinado)

Painel Web Administrativo

- Framework: React 18 + TypeScript + Vite
- UI: Tailwind CSS + shadcn/ui
- Estado: Zustand + React Query (TanStack)
- Deploy: Vercel ou Netlify

2.3 Diagrama de Fluxo (Máquina de Estados da Transação)

Estado	Descrição	Transição
IDLE	Smart POS aguardando pedido	Bartender lança itens
CART_OPEN	Itens adicionados ao carrinho	Cliente apresenta pulseira
NFC_READING	Lendo UID da pulseira	UID capturado com sucesso
VALIDATING	Servidor verifica saldo	Saldo suficiente
PENDING_PAYMENT	Reserva de saldo feita	Confirmação do cliente
PROCESSING	Débito sendo executado	Transação confirmada
FISCAL	Emitindo NFC-e	XML autorizado pela SEFAZ
COMPLETED	Venda finalizada	Retorno ao IDLE
FAILED	Erro (saldo insuf., NFC inválido, offline)	Reversão e alerta

3. Integração NFC / Pulseira

3.1 Como Funciona o NFC Cashless

A pulseira NFC (ou cartão) é apenas um código único (UID). Todo o saldo e histórico ficam no servidor, não na pulseira. O leitor NFC apenas captura o UID e consulta o servidor.

3.2 Hardware NFC Recomendado

Dispositivo	Padrão NFC	Faixa de Preço	Observação
Pulseiras MIFARE Classic 1K	ISO 14443-A	R\$ 3-8 por unidade	Mais barata, funciona na maioria das maquininhas
Pulseiras NTAG213	ISO 14443-A	R\$ 5-12 por unidade	Recomendada — mais compatível
Cartões NFC plásticos	ISO 14443-A/B	R\$ 2-6 por unidade	Alternativa ao bracelete
Leitor USB ACR122U	ISO 14443/15693	R\$ 150-200	Para totens de recarga

3.3 Código Android para Leitura NFC

No AndroidManifest.xml:

```
<uses-permission android:name="android.permission.NFC" />
<uses-feature android:name="android.hardware.nfc" android:required="true" />
```

Na Activity principal (NfcAdapter):

```
val nfcAdapter = NfcAdapter.getDefaultAdapter(this)
nfcAdapter.enableReaderMode(this, { tag ->
    val uid = tag.id.toHexString() // UID da pulseira
    viewModel.processNfcUid(uid)   // Consulta o servidor
}, NfcAdapter.FLAG_READER_NFC_A or NfcAdapter.FLAG_READER_NFC_B, null)
```

3.4 API de Consulta NFC (Backend)

Endpoint de consulta de saldo:

```
GET /api/v1/cards/{uid}/status
Resposta: { uid, status, saldo_disponivel, nome_cliente, historico_recente }
```

Endpoint de reserva de saldo (antes de debitar):

```
POST /api/v1/cards/{uid}/reserve
Body: { valor, transacao_id, pdv_id }
Resposta: { reserva_id, saldo_anterior, saldo_apos, expira_em }
```

Endpoint de confirmação de pagamento:

```
POST /api/v1/cards/{uid}/confirm
Body: { reserva_id, itens[], pdv_id }
Resposta: { transacao_id, status, comprovante_url }
```

IMPORTANTE: Usar padrão de reserva (reserve + confirm) evita duplo débito em caso de falha de rede. O saldo é travado por 60 segundos durante a reserva.

4. Integração com Maquininhas de Cartão

As maquininhas Smart POS rodam Android e permitem instalar APKs personalizados. Cada adquirente fornece um SDK próprio para processar pagamentos com cartão (crédito/débito) além do pagamento via pulseira NFC.

4.1 Maquininhas Compatíveis e SDKs

Adquirente	SDK / API	Modelo Smart POS	Documentação
Pague Seguro (Stone)	Stone SDK Android	Pinpad S920 / A920 / Ingenico APOS	developer.stone.com.br
Stone	Stone SDK Android	PAX A920 / Sunmi P2	developer.stone.com.br
Cielo	Cielo LIO SDK	Cielo LIO V3 / LIO+	developer.cielo.github.io
PagSeguro	PlugPag SDK	PAGBANK Smart A910/A920	dev.pagbank.uol.com.br
Mercado Pago	MP Point SDK	Ingenico MOVE5000 / Smart	mercadopago.com.br/developers
Getnet (Santander)	Getnet SDK	Gertec GPOS700	developers.getnet.com.br
Rede (Itaú)	Rede SDK / TEF	Ingenico ICT220	rede.com.br/desenvolvedores
SumUp	SumUp Android SDK	Air Plus (via Bluetooth)	developer.sumup.com
Vero (Banrisul)	Vero SDK	Gertec GPOS700 / PAX	developers.vero.com.br

4.2 Arquitetura de Abstração de Pagamentos

Para suportar todos os adquirentes sem reescrever o código, usar o padrão Strategy:

```
// Interface comum para todos os SDKs
interface PaymentGateway {
    fun processPayment(amount: Long, type: PaymentType): PaymentResult
    fun cancelPayment(transactionId: String): Boolean
    fun getTransactionStatus(id: String): TransactionStatus
}

// Implementações para cada adquirente
class StoneGateway : PaymentGateway { ... }
class CieloGateway : PaymentGateway { ... }
class PagSeguroGateway : PaymentGateway { ... }
class MercadoPagoGateway : PaymentGateway { ... }

// Factory detecta o adquirente automaticamente
class PaymentGatewayFactory {
    fun create(): PaymentGateway = when (detectAcquirer()) {
```



```
        STONE -> StoneGateway()
        CIELO -> CieloGateway()
        PAGSEGURO -> PagSeguroGateway()
        else -> GenericTEFGateway()
    }
}
```

4.3 Integração Stone / Pague Seguro (Prioridade)

Dependências no build.gradle:

```
implementation 'br.com.stone:stone-sdk:3.3.0'
implementation 'br.com.stone:stone-pos:1.0.0'
```

Inicialização do SDK:

```
Stone.setAppName("PDV Bar")
Stone.setEnvironment(Environment.PRODUCTION)
StoneStart.execute(this) { success ->
    if (success) activateTerminal(stoneCode)
}
```

Processar pagamento com cartão:

```
val transaction = TransactionObject().apply {
    amount = valorEmCentavos
    typeOfTransaction = TypeOfTransactionEnum.DEBIT // ou CREDIT
    isCapture = true
}
PaymentAction().charge(activity, transaction, listener)
```

4.4 Integração Cielo LIO

Dependência:

```
implementation 'com.cielo.lio:order-manager:1.28.0'
```

Criar pedido e cobrar:

```
val order = orderManager.createDraftOrder("PDV-${System.currentTimeMillis()}")
order.addItem(sku, "Produto", preco, quantidade, "UN")
orderManager.placeOrder(order)
orderManager.checkoutOrder(order.id, valor, listener)
```

4.5 Integração PagSeguro PlugPag

Dependência:

```
implementation 'br.com.uol.pagseguro:plugpag:2.0.0'
```

Cobrar:

```
val plugPag = PlugPag(context)
val paymentData = PlugPagPaymentData(
    type = PlugPag.TYPE_DEBITO,
    amount = valorEmCentavos,
    installmentType = PlugPag.INSTALLMENT_TYPE_A_VISTA,
    installments = 1
)
plugPag.doAsyncPayment(paymentData, listener)
```

5. Backend API (Node.js + NestJS)

5.1 Schema do Banco de Dados (Supabase / PostgreSQL)

Tabelas principais:

Tabela	Colunas Principais	Propósito
events	id, nome, data_inicio, data_fim, status	Eventos/festas cadastrados
cards	id, uid_nfc, event_id, cliente_nome, saldo, status	Pulseiras/cartões vinculados
recharges	id, card_id, valor, forma_pagamento, operador_id, criado_em	Histórico de recargas
sessions	id, pdv_id, operador_id, abertura, fechamento, total_vendas	Sessões do PDV
orders	id, session_id, card_id, status, total, criado_em	Pedidos/comandas
order_items	id, order_id, produto_id, qtd, preco_unit, total	Itens dos pedidos
products	id, event_id, nome, preco, categoria, ativo	Cardápio de produtos
transactions	id, order_id, card_id, valor, tipo, gateway, status	Transações financeiras
saldo_reserves	id, card_id, valor, transacao_id, expira_em, status	Reservas de saldo (prevent duplo débito)
nfce_documents	id, order_id, chave_acesso, xml, danfe_url, status	Documentos fiscais emitidos
pdv_terminals	id, serial, adquirente, modelo, status, ultimo_sync	Terminais PDV cadastrados
sync_queue	id, pdv_id, operacao, payload, tentativas, sincronizado_em	Fila de sincronização offline

5.2 Endpoints da API

Gestão de Cartões NFC

- GET /api/v1/cards/{uid}/status — Consulta saldo e status
- POST /api/v1/cards — Vincular novo cartão a evento
- POST /api/v1/cards/{uid}/recharge — Recarregar saldo
- POST /api/v1/cards/{uid}/reserve — Reservar saldo para venda
- POST /api/v1/cards/{uid}/confirm — Confirmar débito
- POST /api/v1/cards/{uid}/refund — Devolução de saldo
- GET /api/v1/cards/{uid}/history — Histórico de transações

Pedidos e Vendas

- POST /api/v1/orders — Criar pedido

- POST /api/v1/orders/{id}/items — Adicionar itens
- POST /api/v1/orders/{id}/checkout — Finalizar e cobrar
- GET /api/v1/orders/{id} — Status do pedido

Gestão do PDV

- POST /api/v1/pdv/sessions/open — Abrir sessão do caixa
- POST /api/v1/pdv/sessions/close — Fechar caixa com totais
- POST /api/v1/pdv/sync — Sincronizar transações offline
- GET /api/v1/pdv/{id}/status — Status do terminal

Relatórios e Admin

- GET /api/v1/reports/sales — Relatório de vendas
- GET /api/v1/reports/products — Produtos mais vendidos
- GET /api/v1/events/{id}/dashboard — Dashboard do evento em tempo real

5.3 Exemplo de JSON de Venda (Conforme Documentação)

```
{
  "transacao_id": "uuid-v4",
  "pdv_id": "PDV-001",
  "card_uid": "04:A3:B2:11:22:33",
  "data_hora": "2026-07-08T20:05:30Z",
  "itens": [
    { "produto_id": 101, "qtd": 2, "preco_unit": 15.00 },
    { "produto_id": 205, "qtd": 1, "preco_unit": 8.50 }
  ],
  "pagamento": {
    "forma": "nfc_pulseira",
    "valor_total": 38.50
  },
  "fiscal": {
    "emitir_nota": true,
    "cliente_cpf": null
  }
}
```

6. Estratégia Offline-First

O PDV nunca pode parar se a internet cair. A estratégia é: todo pedido é salvo localmente primeiro, e sincronizado com o servidor quando a conexão for restabelecida.

6.1 Outbox Pattern (Android + Room)

```
// Toda operação vai para a fila local primeiro
@Entity(tableName = "sync_queue")
data class SyncQueueItem(
    @PrimaryKey val id: String,
    val operacao: String,          // "create_order", "nfc_debit", etc.
    val payload: String,          // JSON serializado
    val tentativas: Int = 0,
    val sincronizado: Boolean = false,
    val criado_em: Long = System.currentTimeMillis()
)

// Worker processa a fila quando há conexão
class SyncWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx,
params) {
    override suspend fun doWork(): Result {
        val items = db.syncQueue().getPending() // FIFO
        items.forEach { item ->
            try {
                api.sync(item) // Envia ao servidor
                db.syncQueue().markSynced(item.id)
            } catch (e: Exception) {
                db.syncQueue().incrementRetries(item.id)
            }
        }
        return Result.success()
    }
}
```

6.2 Regras de Operação Offline

Operação	Offline (sem internet)	Online
Leitura de NFC / saldo	Usa último saldo cacheado (modo degradado)	Consulta servidor em tempo real
Criar pedido	Salva localmente, enfileira sync	Envia imediatamente ao servidor
Débito de saldo	NÃO permitido (risco de duplo débito)	Reserva + confirma no servidor
Pagamento com cartão	Processado pelo SDK da maquininha (funciona offline)	Confirmado no servidor

Emissão de NFC-e	Salva para emitir quando voltar online	Emitte em tempo real via SEFAZ
------------------	--	--------------------------------

Para pagamentos NFC (pulseira), exigir conexão é a decisão mais segura. Configure um timeout de 3 segundos: se não conectar, bloquear a transação NFC e oferecer pagamento com cartão bancário (que funciona offline via SDK).

7. Conformidade Fiscal (NFC-e / SAT)

7.1 Requisitos

- Certificado Digital A1 (arquivo .pfx) ou A3 (hardware token)
- Inscrição estadual no estado de operação
- Credenciais na SEFAZ do estado
- No Amazonas: NFC-e (NFC-e AM) — homologação em ambiente de teste antes de produção

7.2 Bibliotecas/SDKs de Emissão

Solução	Tipo	Custo	Observação
Focus NFe / NFe.io	API REST (SaaS)	~R\$0,10 a R\$0,30 por nota	Mais fácil de integrar, recomendado
ACBR (ACBrNFCe)	Biblioteca Delphi/componente	Gratuito (open source)	Mais complexo, controle total
Plug NFe	API REST	Plano mensal	Boa documentação
eNotas	API REST	Plano mensal	Suporte ao Amazonas

7.3 Exemplo de Integração com Focus NFe

```
// Node.js – emitir NFC-e via Focus NFe API
const axios = require('axios');

const nfce = await axios.post('https://api.focusnfe.com.br/v2/nfce', {
  numero_emissao: pedido.numero,
  data_emissao: new Date().toISOString(),
  consumidor_final: 1,
  itens: pedido.itens.map(item => ({
    numero_item: item.index,
    codigo_produto: item.produto_id,
    descricao: item.nome,
    quantidade_comercial: item.qtd,
    valor_unitario_comercial: item.preco,
    origem_mercadoria: 0,
    situacao_tributaria: '102',
  })),
  formas_pagamento: [{ forma_pagamento: '99', valor: pedido.total }],
}, { auth: { username: process.env.FOCUS_TOKEN, password: '' } });
```

8. Painel Administrativo Web

8.1 Telas Principais

Tela	Funcionalidades
Dashboard do Evento	Vendas em tempo real, cartões ativos, saldo total circulando
Gestão de Cartões	Vincular/bloquear pulseiras, ver histórico, devolver saldo
Recarga de Saldo	Totem virtual: recarga via PIX, cartão, dinheiro
Cardápio / Produtos	CRUD de produtos, preços, categorias, disponibilidade
Relatórios de Vendas	Filtros por PDV, período, produto, operador
Gestão de PDVs	Status dos terminais, operadores, sessões abertas
Fiscal	Listar NFC-e emitidas, reemitir DANFES, status SEFAZ
Fechamento de Evento	Totais, devoluções pendentes, relatório final

8.2 Funcionalidade de Recarga (Totem)

O totem de recarga pode ser uma tablet com o painel web aberto, ou a própria maquininha Smart POS em modo de recarga. Fluxo:

- Operador lembra o cartão NFC no leitor
- Sistema exibe o saldo atual do cliente
- Operador digita o valor da recarga
- Cliente paga com cartão bancário ou PIX na maquininha
- Sistema confirma pagamento e credita o saldo na pulseira

9. Plano de Desenvolvimento por Fases

Estimativa total: 16 a 20 semanas para um desenvolvedor sênior ou equipe pequena.

FASE 1: FUNDACAO E BACKEND CORE (3-4 semanas)

1	Configurar projeto NestJS + TypeScript no GitHub
2	Criar banco PostgreSQL no Supabase com todas as tabelas do schema
3	Implementar sistema de autenticação JWT (login de operadores e admins)
4	CRUD de eventos, produtos e terminais PDV
5	Endpoints de consulta e gestão de cartões NFC
6	Sistema de reserva de saldo (reserve + confirm + rollback)
7	Fila de sincronização (sync_queue) com processamento FIFO
8	Testes unitários dos endpoints críticos (Jest)
9	Deploy do backend em produção (Railway ou Render)

FASE 2: APLICATIVO SMART POS (ANDROID) (5-6 semanas)

1	Criar projeto Android (Kotlin + Jetpack Compose) no Android Studio
2	Implementar Room Database local com todas as entidades
3	Tela de login e abertura de sessão do caixa
4	Tela principal: montagem de pedido (grade de produtos com fotos)
5	Implementar NfcAdapter para leitura de pulseiras (retorna UID)
6	Integrar Stone SDK (Pague Seguro) para pagamento com cartão
7	Implementar PaymentGatewayFactory com suporte a Cielo LIO e PagSeguro
8	Fluxo completo NFC: ler UID -> reservar saldo -> confirmar -> imprimir
9	Outbox Pattern: salvar offline e sincronizar com WorkManager
10	Tela de fechamento de caixa com totais do dia
11	Gerar APK assinado e testar nas maquininhas físicas

FASE 3: PAINEL ADMIN WEB (3-4 semanas)

1	Criar projeto React 18 + TypeScript + Vite
2	Tela de login e seleção de evento
3	Dashboard em tempo real (Supabase Realtime) com gráficos
4	Tela de gestão de pulseiras/cartões com busca e filtros
5	Tela de recarga de saldo (simular totem)
6	CRUD completo de cardápio e produtos
7	Relatórios de vendas com exportação CSV/PDF
8	Tela de fechamento de evento e devoluções

9	Tela de status dos terminais PDV
10	Deploy na Vercel com domínio personalizado

FASE 4: FISCAL E CONFORMIDADE (2-3 semanas)

1	Integrar API Focus NFe (ou eNotas) para emissão de NFC-e
2	Configurar certificado digital A1 no servidor
3	Homologar emissão no ambiente de testes da SEFAZ-AM
4	Implementar fila de emissão (tentar emitir imediatamente, retentar se falhar)
5	Tela de consulta e reemissão de DANFE no painel admin
6	Configurar emissão por CPF opcional (Nota do Consumidor)
7	Testes de conformidade e validação fiscal

FASE 5: TESTES, SEGURANÇA E GO-LIVE (3-4 semanas)

1	Testes de carga: simular 100+ transações simultâneas
2	Testes de ponta a ponta com pulseiras NFC reais
3	Testes de failover: desligar internet durante transação
4	Implementar idempotência em todos os endpoints (evitar dupla cobrança)
5	Auditoria de segurança: SQL injection, rate limiting, CORS
6	Criptografar dados sensíveis (UIDs de pulseiras, CPFs)
7	Monitoramento com Sentry (erros) e logs estruturados
8	Treinamento dos operadores de caixa
9	Evento piloto (teste em produção controlada)
10	Ajustes pós-piloto e documentação final

10. Estrutura de Pastas do Projeto

10.1 Repositório (Monorepo)

```
pdv-cashless/
├── backend/                                # NestJS API
│   ├── src/
│   │   ├── cards/                        # Módulo de cartões NFC
│   │   ├── orders/                      # Módulo de pedidos
│   │   ├── payments/                   # Módulo de pagamentos
│   │   ├── fiscal/                     # Módulo NFC-e
│   │   ├── events/                    # Módulo de eventos
│   │   └── sync/                       # Módulo de sincronização
│   └── package.json
├── android/                              # App Smart POS (Kotlin)
│   ├── app/src/main/
│   │   ├── java/.../
│   │   │   ├── ui/                     # Telas Jetpack Compose
│   │   │   ├── data/                  # Room DAO e Entidades
│   │   │   ├── domain/               # Use Cases / ViewModels
│   │   │   └── gateway/              # SDKs de pagamento (Stone, Cielo...)
│   │   └── res/                      # Layouts, strings, icos
├── web-admin/                            # Painel React
│   ├── src/
│   │   ├── pages/                     # Páginas do painel
│   │   ├── components/               # Componentes reutilizáveis
│   │   └── services/                 # Chamadas à API
├── docs/                                # Documentação técnica
└── README.md
```

11. Variáveis de Ambiente

```
# Backend (.env)
DATABASE_URL=postgresql://user:pass@host:5432/pdv
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_KEY=eyJ...
JWT_SECRET=super_secret_key
FOCUS_NFE_TOKEN=token_foco_nfe
REDIS_URL=redis://localhost:6379

# Android (local.properties)
STONE_CODE=stone_activation_code
API_BASE_URL=https://api.seudominio.com.br
API_TIMEOUT=5000
```

12. Prompts Prontos para o Cursor IDE

Copie e cole estes prompts no Cursor para acelerar o desenvolvimento:

12.1 Criar Estrutura Inicial do Backend

"Crie um projeto NestJS com TypeScript para um sistema PDV cashless. Configure: PostgreSQL com TypeORM, Supabase para realtime, JWT auth, e os módulos: cards (NFC), orders, payments, fiscal, events e sync. Use a estrutura de pastas que vou descrever..."

12.2 Implementar Reserva de Saldo

"Implemente no NestJS um serviço de reserva de saldo para cartões NFC com os métodos: reserveSaldo(uid, valor, transacaoId), confirmSaldo(reservId) e rollbackSaldo(reservId). A reserva deve expirar em 60 segundos automaticamente. Use PostgreSQL transactions para garantir atomicidade."

12.3 App Android - Leitura NFC

"Crie uma Activity Android em Kotlin que usa NfcAdapter.enableReaderMode para ler o UID de pulseiras NFC (NTAG213 e MIFARE Classic). Ao detectar uma tag, exiba o UID em hexadecimal e chame um ViewModel que consulta a API REST em https://api.seudominio.com.br/api/v1/cards/{uid}/status."

12.4 Integrar Stone SDK

"Integre o Stone SDK (br.com.stone:stone-sdk:3.3.0) em um projeto Android Kotlin. Crie uma classe StoneGateway que implementa a interface PaymentGateway com os métodos processPayment(amount: Long, type: PaymentType) e cancelPayment(transactionId: String). Inclua inicialização do SDK e tratamento de erros."

12.5 Painel React - Dashboard em Tempo Real

"Crie uma tela de dashboard em React 18 + TypeScript que usa Supabase Realtime para mostrar em tempo real: total de vendas do dia, número de transações, cartões ativos e saldo total circulando. Use recharts para os gráficos e atualize a cada nova venda via WebSocket."

13. Estimativa de Custos de Infraestrutura

Serviço	Plano	Custo Mensal (R\$)
Supabase (banco + realtime)	Pro (\$25/mês)	~R\$ 140
Railway / Render (backend)	Starter (\$5/mês)	~R\$ 28
Vercel (painel web)	Pro (\$20/mês)	~R\$ 112
Focus NFe (fiscal)	0,20 por nota	Varia por volume
Sentry (monitoramento)	Free tier	Gratuito (até 5k erros/mês)
Redis (Upstash)	Free tier	Gratuito (até 10k req/dia)
Total base mensal		~R\$ 280 + custo fiscal

As pulseiras NFC (NTAG213) custam R\$ 5-12 cada no atacado. Para um evento de 500 pessoas, custo de pulseiras: R\$ 2.500-6.000. Pulseiras são reutilizáveis entre eventos.

14. Próximos Passos Imediatos

12. Criar repositório no GitHub com estrutura de monorepo
13. Criar conta no Supabase e configurar o schema das tabelas
14. Criar conta de desenvolvedor na Stone (Pague Seguro) para obter credenciais de sandbox
15. Solicitar acesso ao SDK da Cielo LIO (developercielo.github.io)
16. Criar conta na Focus NFe para testes de NFC-e em homologação
17. Comprar 5-10 pulseiras NTAG213 para testes de NFC
18. Configurar ambiente de desenvolvimento: Android Studio + Node.js 20
19. Começar pela Fase 1: backend core (endpoints NFC + reserva de saldo são os mais críticos)

Documento gerado por Claude — Base: documentação técnica fornecida + melhores práticas do ecossistema Android/Node.js brasileiro